



ALBUKHARY INTERNATIONAL UNIVERSITY

SCHOOL OF COMPUTING & INFORMATICS

Year - 2 Semester-1

Course Title : **Data Structure and Algorithm Analysis**

Course Code : **CCC 2133**

Assignment 2

Arithmetic Evaluation - Infix to Postfix Conversion Using Stack and Queue

Group C

Name	Student ID
Mohammed Oussama Chakeur	AIU21102120
Kyi Nuu Phyu Sin	AIU23102105
Hnin Eaindra Htun	AIU23102124
MD Parvej Ahmed Rafi	AIU23102301

Table of Contents

Problem Statement.....	2
Explanation of the Algorithm.....	3
1. Infix to Postfix Conversion.....	3
2. Postfix Evaluation.....	3
Sample Inputs and Outputs.....	4
Test Cases and Their Results.....	4
Sample Output.....	6
Time Complexity Analysis.....	6
a. Infix to Postfix Conversion.....	6
b. Postfix Evaluation.....	7
Space Complexity Analysis.....	7
a. Infix to Postfix Conversion.....	7
b. Postfix Evaluation.....	8
Challenges faced and how they were overcome.....	8

Problem Statement

Infix notation, where operators are written between operands, is commonly used in arithmetic expressions. However, evaluating such expressions directly is computationally challenging due to operator precedence and parentheses. Postfix notation, where operations follow their operands, simplifies evaluation but requires conversion from infix form.

This project addresses these challenges by designing and implementing a program that:

1. Converts infix expressions to postfix notation using stack and queue data structures.
2. Evaluates the resulting postfix expressions to provide the final result.
3. Handles edge cases such as invalid input, mismatched parentheses, and operator precedence effectively.

The program must use singly linked lists to implement the stack and queue, simulating real-world applications of data structures in computational problems.

Explanation of the Algorithm

1. Infix to Postfix Conversion

The infix notation is a common way to write arithmetic expressions, where operators are placed between operands (e.g., $a + b$). Postfix notation, on the other hand, places operators after operands (e.g., $a b +$). The conversion from infix to postfix uses a stack and follows these rules:

1. **Operands:** Append directly to the postfix queue.
2. **Operators:** Push onto the stack. If the operator at the top of the stack has higher or equal precedence, pop it to the queue.
3. **Parentheses:**
 - Push opening parentheses to the stack.
 - On encountering a closing parenthesis, pop and append operators to the queue until an opening parenthesis is found.
4. **End of Expression:** Append remaining operators from the stack to the queue.

2. Postfix Evaluation

Once the postfix expression is generated, it can be evaluated using a stack:

1. **Operands:** Push onto the stack.
2. **Operators:** Pop the top two operands from the stack, apply the operator, and push the result back onto the stack.
3. **Final Result:** After processing all tokens, the stack will contain the result of the expression.

Sample Inputs and Outputs

Infix Expression	Postfix Expression	Evaluation Result
$a+b*(c^d-e)^{(f+g*h)}-i$	$a\ b\ c\ d\ ^\ e\ -\ f\ g\ h\ *\ +\ ^\ *\ +\ i\ -$	Cannot Evaluate (Contains Variables)
$A+B*(C+D)-E/F*G+H$	$A\ B\ C\ D\ +\ *\ +\ E\ F\ /\ G\ *\ -\ H\ +$	Cannot Evaluate (Contains Variables)
$3+2*(6^2-4)$	$3\ 2\ 6\ 2\ ^\ 4\ -\ *\ +$	67
$500+5^3-(10+56)*9$	$500\ 5\ 3\ ^\ +\ 1\ 0\ 5\ 6\ +\ 9\ *\ -$	-99
$3+5*(5/5)-2^2$	$3\ 5\ 5\ 5\ /\ *\ +\ 2\ 2\ ^\ -$	4

Test Cases and Their Results

Cases	Prefix Expression (Input)	Expected Output (Postfix)	Postfix Evaluation	Test Result
Test Case 1 (Valid Numeric Expression)	$3+2*(6^2-4)$	3 2 6 2 ^ 4 - * +	67	Passed
Test Case 2 (Valid Variable Expression)	$a+b*(c-d)$	a b c d - * +	Cannot Evaluate (Contains Variables)	Passed
Test Case 3 (Mismatched Parentheses)	$(1+2)*(3+4$	Error - Invalid Expression:Misma tched Parentheses		Passed
Test Case 4 (Empty Input)	""	Error - Invalid Character in Expression		Passed
Test Case 5 (Complex Nested Parentheses)	$((2+3)*5)-(7/(1+1))$	2 3 + 5 * 7 1 1 + / -	22	Passed

Sample Output

```
---jGRASP exec: java InfixToPostfix
Reading expressions from file...

Infix Expression: a+b*(c^d-e)^(f+g*h)-i
Postfix Expression: a b c d ^ e - f g h * + ^ * + i -
Postfix Evaluation: Cannot evaluate postfix expressions with variables or non-numeric operands.

Infix Expression: A+B*(C+D)-E/F*G+H
Postfix Expression: A B C D + * + E F / G * - H +
Postfix Evaluation: Cannot evaluate postfix expressions with variables or non-numeric operands.

Infix Expression: 3+2*(6^2-4)
Postfix Expression: 3 2 6 2 ^ 4 - * +
Postfix Evaluation: 67

Infix Expression: 500+5^3-(10+56)*9
Postfix Expression: 5 0 0 5 3 ^ + 1 0 5 6 + 9 * -
Postfix Evaluation: -99

Infix Expression: 3+5*(5/5)-2^2
Postfix Expression: 3 5 5 5 / * + 2 2 ^ -
Postfix Evaluation: 4

Now you can input your own expressions (type 'exit' to quit):
▶ Enter an infix expression: [
```

Here, we construct the code to read the input from the .txt file. Additionally, the system can work for the user input also.

Time Complexity Analysis

a. Infix to Postfix Conversion

1. Input Traversal:

- Each character (operator, operand, or parenthesis) in the infix expression is processed exactly once.
- Time Complexity:** $O(n)$, where n is the length of the infix expression.

2. Stack Operations:

- For operators and parentheses, push and pop operations are $O(1)$.
- Each operator and parenthesis can be pushed and popped exactly once.
- Time Complexity:** $O(n)$.

3. Queue Operations:

- Each operand and operator is added to the postfix queue using enqueue operations, which are $O(1)$.
- This happens once for each character.
- Time Complexity:** $O(n)$.

4. Overall Time Complexity:

- **Best Case:** $O(n)$, simple expressions like $a+b+c$ with no parentheses.
- **Worst Case:** $O(n)$, complex expressions like $(a+b)*(c+d)$, where every character is processed and involves stack operations.
- All operations (input traversal, stack and queue operations) are linear in nature.
- **Final Time Complexity:** $O(n)$.

b. Postfix Evaluation

1. Traversal of Postfix Expression:

- Each token (operand or operator) in the postfix expression is processed exactly once.
- **Time Complexity:** $O(n)$, where n is the length of postfix expression.

2. Stack Operations:

- Each operand is pushed onto the stack, and for every operator, two operands are popped, and the result is pushed back.
- Push and pop operations are $O(1)$, and each token is processed once.
- **Time Complexity:** $O(n)$.

3. Overall Time Complexity:

- **Final Time Complexity:** $O(n)$.

Space Complexity Analysis

a. Infix to Postfix Conversion

1. Stack for Operators:

- In the worst case, all operators and parentheses are stored in the stack simultaneously.
- **Space Complexity:** $O(n)$, where n is the length of the infix expression.

2. Queue for Postfix Expression:

- The postfix queue stores the entire postfix expression, which can be as long as the input expression.
- **Space Complexity:** $O(n)$.

3. **Overall Space Complexity:**
 - **Final Space Complexity:** $O(n)$.

b. Postfix Evaluation

1. **Stack for Operand Storage:**
 - In the worst case, all operands are stored on the stack simultaneously (eg., 12345 in postfix).
 - **Space Complexity:** $O(n)$, where n is the length of the postfix expression.
2. **Overall Space Complexity:**
 - **Final Space Complexity:** $O(n)$.

Challenges faced and how they were overcome

1. Handling Operator Precedence

- **Challenge:** Ensuring the correct precedence of operators like $+$, $-$, $*$, $/$, and $^$ during the conversion from infix to postfix.
- **Solution:** A method **precedence()** was implemented to define and return precedence levels for each operator. The algorithm utilized this method during parsing to correctly order operators in the postfix output.

2. Managing Parentheses

- **Challenge:** Correctly handling opening and closing parentheses to group sub-expressions and ensure the validity of expressions.
- **Solution:** Opening parentheses were pushed onto the stack, and when a closing parenthesis was encountered, elements were popped from the stack and added to the postfix output until the matching opening parenthesis was found. An error was raised if mismatched parentheses were detected.

3. Invalid Characters in Input

- **Challenge:** Preventing invalid or unsupported characters in the infix expression from causing errors or undefined behavior.
- **Solution:** A validation mechanism was implemented to check if each character was a valid operand, operator, or parenthesis. Invalid characters triggered an exception with a meaningful error message for the user.

4. Converting Expression Strings

- **Challenge:** Parsing infix expressions with a mix of operators, operands, and parentheses.
- **Solution:** Each character in the input string was processed in sequence, with operands enqueued, operators pushed onto a stack, and parentheses handled explicitly.

5. Evaluating Postfix Expressions

- **Challenge:** Safely evaluating postfix expressions, particularly when encountering division by zero or incorrect numeric input.
- **Solution:** The postfix evaluation function used a stack to compute the result incrementally, applying operators to the operands in the correct order. Edge cases, like division by zero, were caught and handled with appropriate error messages.

6. Managing File Input and Missing Files

- **Challenge:** Ensuring the program could read infix expressions from an external file (**expressions.txt**) and handling scenarios where the file was missing or misplaced.
- **Solution:** File operations were wrapped in a **try-catch** block to handle **FileNotFoundException**. Users were informed if the file was not found and guided to place it in the correct directory.

7. Queue and Stack Implementation

- **Challenge:** Implementing custom queue and stack classes to handle intermediate results during the conversion and evaluation process.
- **Solution:** Simple linked list-based implementations of **Stack** and **Queue** were created, with methods for push, pop, enqueue, and dequeue. Additional methods like **peek()** and **isEmpty()** were added to ensure robust functionality.

8. Mixed Numeric and Variable Expressions

- **Challenge:** Distinguishing between numeric and variable-based expressions, as variable-based expressions cannot be directly evaluated.
- **Solution:** A check was added to skip evaluation if the postfix expression contained variables or non-numeric operands. A clear message was displayed to the user, explaining why the evaluation was skipped.

9. User Interaction for Custom Inputs

- **Challenge:** Allowing users to input custom infix expressions interactively and ensuring the program handled both numeric and variable-based expressions gracefully.
- **Solution:** An interactive loop was added to accept user inputs, convert them to postfix, and evaluate them if possible. The program handled invalid inputs gracefully by catching exceptions and displaying appropriate error messages.